

# ZLC FPGA Pulse Streamer 手册

---

Artix-7 35T 边沿流送器 / 1-tick 预取 / 无限流式扫描 / JTAG-to-AXI

---

RTL、1-tick 预取、双 bank 流式、仿射扫描引擎、模拟总线 DAC、编译上传流  
程、资源预算

2026-06-11

# Contents

|          |                                      |           |
|----------|--------------------------------------|-----------|
| <b>1</b> | <b>写给新读者</b>                         | <b>1</b>  |
| 1.1      | 这块硬件是什么                              | 1         |
| 1.2      | 两句话能力总结                              | 1         |
| 1.3      | 文件                                   | 2         |
| <b>2</b> | <b>架构总览</b>                          | <b>3</b>  |
| 2.1      | 从主机到引脚的一条链                           | 3         |
| 2.2      | CTRL 寄存器邮箱                           | 3         |
| 2.3      | 一行边沿的含义                              | 4         |
| 2.4      | 时钟与对齐                                | 4         |
| <b>3</b> | <b>1-tick 预取流水线</b>                  | <b>5</b>  |
| 3.1      | 为什么需要预取                              | 5         |
| 3.2      | 解法: 深度 FIFO_DEPTH 的连续预取 + 影子         | 5         |
| 3.3      | 背靠背 1-tick 边沿: 前端脉冲实绘                | 5         |
| <b>4</b> | <b>无缝边界: 四个 gapless 重装点</b>          | <b>7</b>  |
| 4.1      | 边界为什么不留缝                             | 7         |
| 4.2      | 扫描点 $k \rightarrow k+1$ 跨界无缝: 前端脉冲实绘 | 7         |
| <b>5</b> | <b>无限流式扫描: 2-bank ping-pong</b>      | <b>9</b>  |
| 5.1      | 窗口 + 流式回填                            | 9         |
| 5.2      | bank_chunk 握手与晚到 STALL               | 9         |
| 5.3      | repeat_forever 流式重扫                  | 10        |
| <b>6</b> | <b>N-slot 仿射扫描引擎</b>                 | <b>11</b> |
| 6.1      | 核心公式                                 | 11        |
| 6.2      | 为什么这是对的取舍                            | 11        |
| <b>7</b> | <b>模拟总线 DAC 引擎</b>                   | <b>12</b> |
| 7.1      | 段表编码                                 | 12        |
| 7.2      | 模拟 DAC ramp 与扫描端点: 前端脉冲实绘            | 12        |

---

|           |                                           |           |
|-----------|-------------------------------------------|-----------|
| <b>8</b>  | <b>三个子系统的深入讨论 . . . . .</b>               | <b>14</b> |
| 8.1       | AXI4 突发上传 . . . . .                       | 14        |
| 8.2       | 逐通道输出延时 (事件调度器 event scheduler) . . . . . | 15        |
| 8.3       | DAC 总线延时与 TTL 完全同源 . . . . .              | 17        |
| 8.4       | 逐通道 clk 输出 (把某通道直接接到时钟) . . . . .         | 17        |
| 8.5       | 两件没有下放进 sequencer 的事 . . . . .            | 18        |
| <b>9</b>  | <b>容量与资源预算 . . . . .</b>                  | <b>19</b> |
| 9.1       | 35T 上解出的容量 . . . . .                      | 19        |
| 9.2       | 64 位 tick 架构评估 (何时需要、怎么改、资源够不够) . . . . . | 19        |
| <b>10</b> | <b>编译与上传流程 . . . . .</b>                  | <b>21</b> |
| 10.1      | build / program / run . . . . .           | 21        |
| 10.2      | 一次上传里发生了什么 . . . . .                      | 21        |

# 1

## 写给新读者

### 1.1 这块硬件是什么

pulse-streamer 是一个**固定 bitstream + 运行时表**的脉冲序列器。烧一次 bitstream 就得到时钟状态机、板上输出引脚、`jtag_axi + axi_bram_ctrl` 数据通路和片上 BRAM；它**不把**某个具体实验脉冲写死进 Verilog。之后每次 `On Pulse`，主机都把当前的 `PulseTableState/PulseSequence` 编译成一张紧凑的程序映像，经 **JTAG-to-AXI** 写进 FPGA 的 BRAM，再用一个 CTRL 寄存器邮箱 (COMMAND/STATUS) 控制引擎运行。

这是**唯一**一套设计：没有变体、没有向后兼容、没有探针控制面、没有逐通道游程引擎、没有加载器 FSM。整条路径就是“主机打包 BRAM 映像 → AXI 写入 → CTRL 邮箱命令 → 边沿表引擎播放”。

#### 目标器件

本设计面向 **Xilinx Artix-7 35T (xc7a35tfgg484-2)**，FGG484 封装。这是一颗 LUT/BRAM 资源有限的小器件，所以所有容量参数都按它来权衡。

近似资源类别：

#### xc7a35tfgg484-2

|                                    |                  |                 |
|------------------------------------|------------------|-----------------|
| logic cells: 33,280                | CLB LUTs: 20,800 | CLB FFs: 41,600 |
| BRAM: 1,800 Kb (50 x 36 Kb RAMB36) | DSP: 90          |                 |

### 1.2 两句话能力总结

- **最小脉冲宽度与分辨率 = 1 个 tick (20 ns)**。背靠背的 1-tick 边沿能逐周期、一拍一个地打出来——靠的是一条连续预取流水线把 BRAM 读延迟藏掉（见第三章）。
- **扫描点数无上限**。常驻一个 2-bank ping-pong 窗口 (4096 点)，主机在游标后面把空出来的 bank 流式回填下一段（见第五章）。配合 N-slot 仿射扫描，**duration 与 DAC 值**（仅这两类可扫）可以在一次连续运行里任意组合地无缝扫描；channel 延时**不是**扫描量，它是加在输出上的逐信号**事件调度延时**（每条 channel / 每条 DAC 总线一个固定值，正/负/零；TTL 与 DAC **同源同范围**，上限均由 32 位字段决定，约  $\pm 42.9$  s——毫秒级 emCCD 延时直接可用，见第六章）。

## 1.3 文件

**fpga/pulse\_streamer/zlc\_edge\_streamer.v** 引擎。全局边沿表 (三块并行 BRAM: tick 32b / coeff 64b / mask 62b, 强制 `READ_LATENCY_B=2`)、深度 `FIFO_DEPTH(=RD_LAT+2=4)` 的连续边沿预取、2-bank ping-pong 扫描窗口、仿射有效 tick MAC、模拟总线 DAC 引擎。

**fpga/pulse\_streamer/zlc\_pulse\_streamer\_top.v** 顶层。`jtag_axi` → `axi_bram_ctrl` → 区段译码的 BRAM (三块并行边沿 + 扫描窗口 + 总线映像) + CTRL 寄存器邮箱, 驱动引擎与板上 62 路输出 + 4x10b DAC 总线。

**fpga/pulse\_streamer/host/image.py** 主机 ↔ FPGA AXI 写契约 + 容量求解器的单一真相源。`pack_program/unpack_program` 打包/解码 BRAM 映像, `solve_capacity` 从器件 RAMB36/LUT 预算推导几何。RTL localparam 与 create-project Tcl 都从它派生。

**fpga/pulse\_streamer/host/engine\_model.py** 逐周期行为模型 (仓库没有 Verilog 仿真器, 这是硬件前的正确性证明)。

**Zou\_lab\_control/neutral\_atom/devices/axi\_session.py** 主机会话 `VivadoAxiStreamerSession`: 常驻 Vivado `hw_axi`, 打包上传 + 驱动 CTRL 邮箱 + 流式回填。

# 2

## 架构总览

### 2.1 从主机到引脚的一条链

真实路径是双电脑结构。控制电脑通过 RPyC 把命令发到 FPGA 电脑上的 server; server 持有一个常驻 Vivado `hw_axi` 会话, 用 JTAG-to-AXI 把程序映像写进 FPGA 的 BRAM, 再写 CTRL 寄存器命令引擎运行。

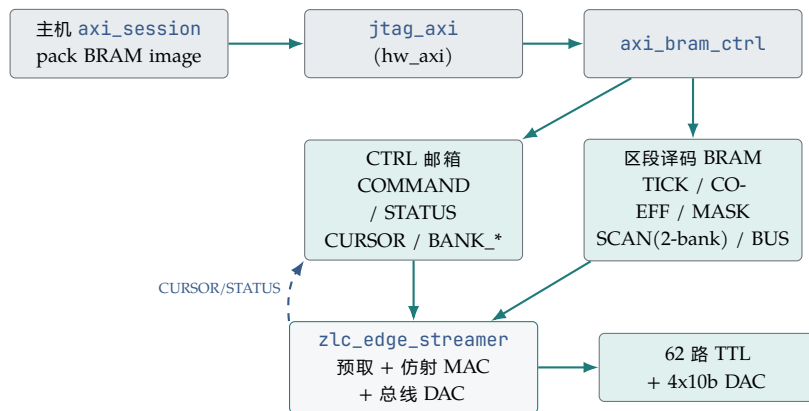


Figure 2.1: 数据通路: 主机经 JTAG-to-AXI 写区段译码的 BRAM 与 CTRL 邮箱; 引擎从 BRAM 播放, 并把 STATUS/CURSOR 回写邮箱供主机轮询/流式回填。

### 2.2 CTRL 寄存器邮箱

主机不碰任何探针; 它只读写一小块 CTRL 寄存器文件 (来自 `host.image.CtrlWords`):

| CTRL mailbox |         |         |                                                     |
|--------------|---------|---------|-----------------------------------------------------|
| COMMAND      | 主机->top | 上升沿     | LOAD(1) / FIRE(2) / RESET(4) / SAFE(8)              |
| STATUS       | top->主机 |         | LOADED(1)/RUNNING(2)/DONE(4)/ERROR(8)/UNDERFLOW(16) |
| PROG_COUNT   |         | 边沿行数    |                                                     |
| SCAN_COUNT   |         | 扫描点总数 N | (可超过常驻窗口)                                           |

```

SCAN_ENABLE / REPEAT_FOREVER
LOOP_START / LOOP_COUNT / LOOP_END_TICK / LOOP_END_LO / LOOP_END_HI
BUS_COUNTS / BANK_SIZE / SLOT_COUNT
CURSOR      top-> 主机    已消费的扫描点数（驱动流式回填）
BANK_READY  主机->top    bit b = bank b 已装好
BANK0_CHUNK / BANK1_CHUNK 主机->top  各 bank 当前装的是哪一段 chunk

```

生命周期: `prepare` (SAFE、上传映像、arm bank、LOAD) / `fire` (FIRE) / `wait_done` (轮询 STATUS、跟随 CURSOR 流式回填) / `safe_state` (SAFE)。COMMAND 字在每个命令前先清 0，制造干净的上升沿。

## 2.3 一行边沿的含义

引擎里保存一张全局边沿表，每行三个字段，分别存在三块**并行的** BRAM 里：

### 每行边沿

```

tick[i]   : 该边沿的基准 FPGA tick (32 bit)           -- TICK BRAM
coeff[i]  : NUM_SLOTS 个定点扫描系数 (NUM_SLOTS*16 bit) -- COEFF BRAM
mask[i]   : 该 tick 上完整的输出状态 (62 bit)         -- MASK BRAM

```

bit 0 对应 `channels[0]`。一行的含义是：**在这个有效 tick，把所有输出切换成这个 mask**——不是逐 channel 命令。三块 BRAM 并行读，所以一次访问拿到一整行边沿，无需位宽填充；`max_edges` 因此可以做大（35T 上 4096 行）。

## 2.4 时钟与对齐

真实硬件默认 50 MHz，一个 tick 是 20 ns。主机在上传前校验所有 period 持续时间、channel 延时、扫描点都对齐到这个 tick 网格 (`1e9 / clock_hz`)。32 位 tick 计数器在 50 MHz 下覆盖约 85.9 秒。同步后的 FIRE 路径会引入一个固定的小启动延迟，但那是常数偏移，不是比例因子；脉冲宽度和延时由 tick 计数决定。

# 3

## 1-tick 预取流水线

### 3.1 为什么需要预取

边沿表放在 block RAM 里, 而 block RAM 是**同步读**: 发出读地址后, 数据要  $RD\_LAT$  个周期后才到。build tcl 用 `zlc_force_latency2` 把边沿 BRAM 强制成  $READ\_LATENCY\_B=2$ , 所以  $RD\_LAT=2$  是确定的。如果天真地“到点了才去读下一行”, 每条边沿之间就至少要空 2 个周期——背靠背的 1-tick 边沿根本打不出来。

### 3.2 解法: 深度 FIFO\_DEPTH 的连续预取 + 影子

引擎维护一个深度  $FIFO\_DEPTH(=RD\_LAT+2=4)$  的“arm” FIFO, 里面装着**接下来要打的几条边沿**。它**每个周期发一次 BRAM 读**, 把读回的边沿压进 FIFO 尾; 当 `time_count` 追上 FIFO 头那条边沿的有效 tick 时, 就在**同一周期**把它的 mask 打到输出、并把它弹出。因为 FIFO 深度比**流水线延迟** (发地址  $o$  数据有效  $= RD\_LAT+1$ , 含寄存的 `edge_raddr` 一拍) 多 1, FIFO 永远不空, 背靠背的 1-tick 边沿就能一拍一个地连续命中。

#### SEED 不变式 (行为模型逼出来的关键)

在每个边界, 从“第一条还没输出的边沿”开始**播种 FIFO\_DEPTH 条影子**, 并且边界当拍**不发新读** (占用  $=$  影子数  $\leq$  深度)。4 条影子刚好够给紧跟着的 1-tick 后继留出时间; 3 条影子会少一拍而丢边沿 (这正是真机 emCCD 第二脉冲 40 ms / e7 丢失的根因——`edge_raddr` 是寄存器, 发地址到数据有效是  $RD\_LAT+1$  拍)。这条不变式是逐 tick 镜像仿真“逼”出来的。

### 3.3 背靠背 1-tick 边沿: 前端脉冲实绘

边沿表放在 block RAM(同步读, $RD\_LAT=2$ ), 引擎用一条深度  $FIFO\_DEPTH=RD\_LAT+2=4$  的连续预取流水线把读延迟藏掉: 它**每周发一次 BRAM 读** (周期  $T$  发出的读, 地址在  $T+1$  才到 BRAM 地址口——`edge_raddr` 是寄存器, 数据在  $T+RD\_LAT+1$  落地), 读回的边沿压进 arm FIFO 尾; 当 `time_count` 追上 FIFO 头那条边沿的有效 tick 时, 在**同一周期**把它的 mask 打到输出并弹出。因为 FIFO 深度比流水线延迟 ( $=RD\_LAT+1$ ) 多 1, 弹出与补充始终咬合、FIFO 不空。下图是**前端脉冲绘图器实绘**的引擎输出: 三路通道在相邻 20 ns tick 上背靠背切换, 每个 1-tick 边沿逐拍打出、中间无空拍——这正是 20 ns 的最小脉宽与分辨率。



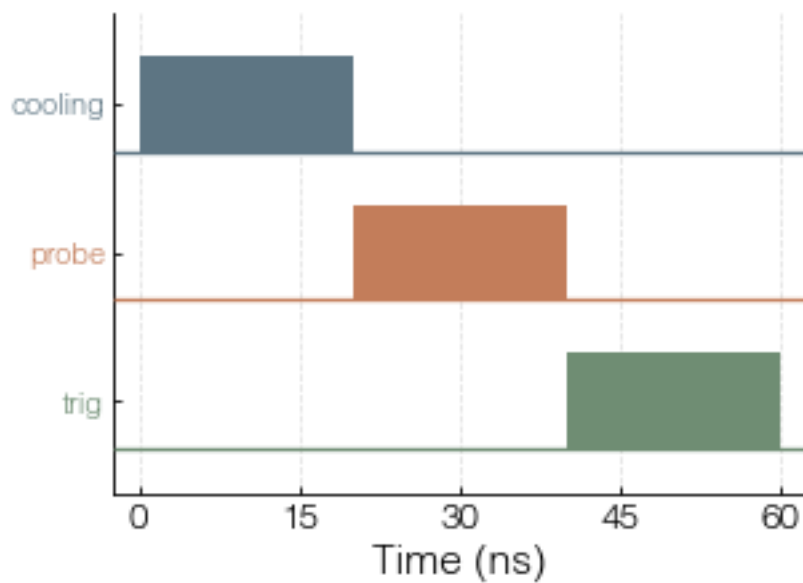


Figure 3.1: 前端脉冲实绘: 三路通道在相邻 20 ns tick 上背靠背切换——引擎的最小脉宽与分辨率就是 1 个 tick; 预取流水线让这些 1-tick 边沿逐拍打出, 中间无空拍。

- 周期  $-2, -1$  是 **arm 阶段** (reset/LOAD 期间): 连发读把前几条边沿的影子预取进 FIFO, 使 `time_count` 从 0 开始时 FIFO 已经有货。
- tick 0: FIFO 头是 A 且 `time_count=tick[A]=0`, **当拍**输出 A 并弹出; 同一拍 C 的数据正好落地补进 FIFO。
- 由于 FIFO 深度  $4 >$  流水线延迟 3 ( $=RD\_LAT+1$ , 含 `edge_raddr` 寄存器一拍), 弹出与补充永远咬合, tick 连续命中 B/C/D, **无空拍**。

这条等价是被证明的

`host.engine_model.rtl_mirror_play` 在读延迟 1/2/3 下逐 tick 等于组合参考播放器 `reference_play`, 并跑了 200 个 fuzz 程序 (含背靠背 1-tick、扫描 1-tick、loop 1-tick)。仓库无 Verilog 仿真器, 这个逐周期镜像就是硬件前最强证据。

# 4

## 无缝边界：四个 gapless 重装点

### 4.1 边界为什么不留缝

引擎只有一个全局边沿指针，所以一次 repeat / loop-rewind / scan-advance / start 只是单周期的指针 + 影子重装。四个 gapless 重装点 (start / loop-rewind / scan-advance / repeat) 在边界用上一节的 SEED 不变式重新播种 FIFO\_DEPTH 条影子，于是点  $k$  的最后一条边沿与点  $k+1$  的第一条边沿在时间上紧挨着，中间没有缝。

### 4.2 扫描点 $k \rightarrow k+1$ 跨界无缝：前端脉冲实绘

仿射扫描把 duration/DAC 值绑到 slot  $s_0 \dots s_N$  (channel 延时不是扫描量)；每个扫描点用新的 slot 向量，边沿的有效 tick =  $\text{base} + (\text{sum coeff} * \text{slot}) \gg \text{FRAC}$  随之平移。下图是前端实绘的同一脉冲在两个扫描点的输出——被扫的中间周期把后面的边沿在硬件里 lockstep 平移。引擎在播放点  $k$  末尾的同一拍，就已按  $k+1$  的新 slot 向量把首批影子重新播种好，所以点  $k$  的末边沿与点  $k+1$  的首边沿在时间上紧挨着、无 gap。

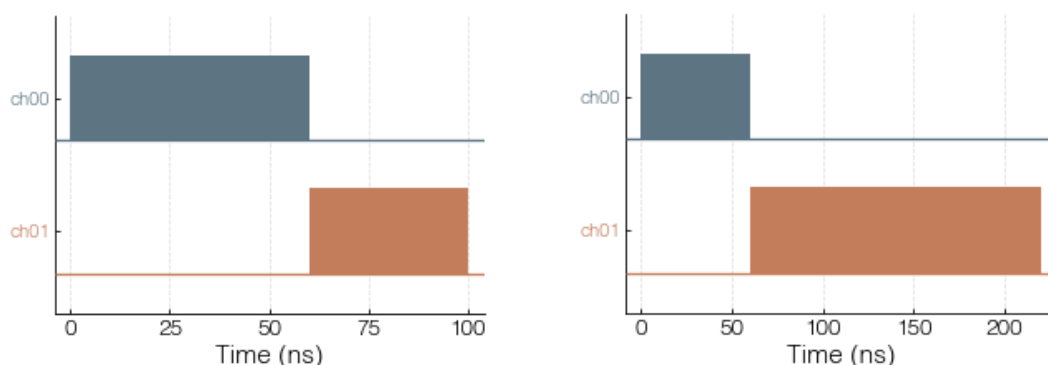


Figure 4.1: 前端脉冲实绘：同一脉冲在两个 scan 点的渲染。被扫的中间周期把后面的边沿在硬件里 lockstep 平移；扫描点之间的切换是无缝的（边界影子重装）。

- 边界处引擎不发新读，只用已经播种好的影子。新 slot 向量在边界当拍生效，所以下一点首条边沿的

有效 tick 已按  $k+1$  的扫描值算好。

- `loop-rewind(repeat bracket)` 走同一套机制: 指针跳回 `loop_start_addr`, 影子按 loop 起点重新播种, 循环接缝同样无缝。下图是前端实绘的硬件重复 loop 循环体 (不展开), 重复之间无缝回绕。

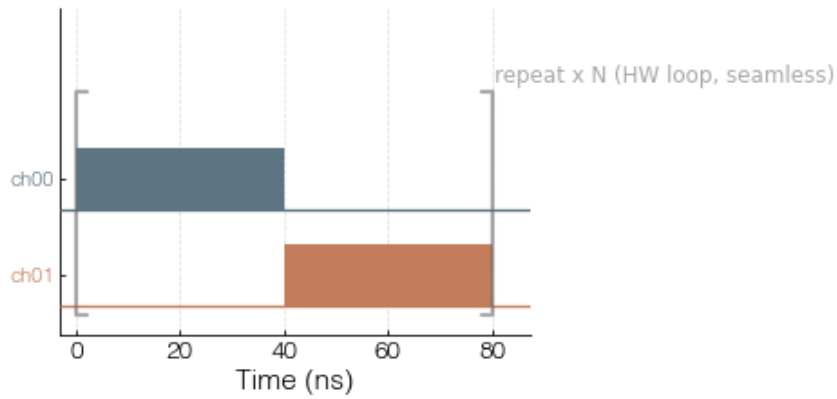


Figure 4.2: 前端脉冲实绘: 硬件重复 loop 的循环体 (不展开)。 `repeat\_forever` 在硬件里无缝回绕, 重复之间不留缝。

`host.engine_model` 用 `streaming_scan_play` 与 1-tick 边界 fuzz 证明了这一点: 跨扫描点 / 跨 loop 的边沿与不分段的参考播放逐 tick 相同。

# 5

## 无限流式扫描：2-bank ping-pong

### 5.1 窗口 + 流式回填

扫描窗口在硬件里是一个 **2-bank ping-pong** ( $BANK\_SIZE=2048$ ，是 2 的幂；两 bank 共 4096 个常驻扫描点)，放在一块 BRAM 里。引擎依次播放点  $0..N-1$ ，用  $((idx/BANK\_SIZE) \bmod 2) \oplus scan\_bank\_base$  选 bank ( $scan\_bank\_base$  实现连续循环 ping-pong，见下“无缝重复扫描”)，并把已消费点数写进 **CURSOR**。主机读 **CURSOR**，把游标已经走过的那个 bank 回填**下一段 chunk**，并更新 **BANK\_READY/BANK\*\_CHUNK**。这样扫描点文件虽然有限，但**大小无上限**。



Figure 5.1: 2-bank ping-pong: 引擎播放当前 bank、推进 CURSOR；主机在游标背后把空出来的 bank 回填下一段 chunk。

### 5.2 bank\_chunk 握手与晚到 STALL

引擎只在 **BANK\_READY** 为真且该 bank 装的是**正确 chunk** ( $BANK*_CHUNK$  与引擎期望的段号一致) 时才推进。如果回填来晚了，引擎**停住** (hold 在当前点，置 **STATUS\_UNDERFLOW**)，**绝不**播放错误或过期的点；主机回填到位后它再继续。这把“晚到”变成一次安全的 hold，而不是数据损坏。

#### streaming refill timeline + late-refill STALL

```
CURSOR (pts done):  0 ..... 2047 | 2048 ..... 4095 | 4096 .....
engine plays bank:  bank0          | bank1          | bank0(chunk2)
BANK_READY         :  b0,b1 ready  | b0 freed      | refill in flight
BANK0_CHUNK        :  0            | (host -> 2)   | 2
host refill bank0 :      [.... writing chunk 2 ....]
                    ^
case A 及时:  chunk2 在游标到 4096 前写好 -> BANK0_CHUNK=2 ready -> 无缝继续
case B 晚到:  游标到 4096 但 bank0 还不是 chunk2
```

```
-> 引擎 STALL (hold 当前点, STATUS_UNDERFLOW=1)
-> 主机写完 chunk2 + BANK0_CHUNK=2 + BANK_READY
-> 引擎自动继续, 不丢点/不串点
```

### 5.3 repeat\_forever 流式重扫

`repeat_forever` 对一个有限（但很大）的流式扫描做反复扫掠：`fire` 时启动一个主机**后台回填线程**，循环把下一段填进空出的 bank。一次扫掠之内是**无缝**的；扫掠与扫掠之间的接缝是一个**短暂的安全 hold**（重新从 chunk 0 播种）。`host.engine_model.streaming_scan_play` 证明了播放序列、CURSOR 推进、以及晚到 STALL 行为。

# 6

## N-slot 仿射扫描引擎

### 6.1 核心公式

每行边沿存一个 base tick 加 NUM\_SLOTS 个定点系数；运行时 FPGA 对每个扫描点计算：

effective tick

```
effective_tick = base_tick + (sum_j coeff_j * slot_j) >>> COEFF_FRAC_BITS
```

其中 `slot_j` 是当前扫描点第  $j$  个 slot 的值（来自扫描点表那一行），`COEFF_FRAC_BITS=8` 是定点小数位数。**已经没有  $x/y$** ——slot 是任意多个命名维度 `s0, s1, ...`，每个绑定一个 duration 或 DAC 值字段（`SCAN_SLOT_KINDS=("duration", "dac")`；channel 延时是固定值，不是扫描量）。这同一个 `effective_tick` 表达式被边沿 tick、loop 终点 tick、以及总线段起止 tick **复用**——这正是为什么 duration 与 DAC 值能任意组合地同时扫描，且只花一条 MAC 的 LUT。

### 6.2 为什么这是对的取舍

spec 最初设想的是逐通道游程编码（per-channel）+ 双 bank 全新存储。我们改成把仿射引擎**泛化**成  $N$  个 slot 的流式扫描点表，原因围绕 35T 的现实：

- 存储复杂度  $O(\text{edges}) + O(\text{points} \times \text{slots})$  已达成 spec 的核心存储目标，但不需要每通道游程的状态机与地址逻辑。
- LUT 最省：一条比较器 + 一条仿射 MAC，而不是 62 个通道 player。
- 可验证：在无 Verilog 仿真器的前提下，仿射方案能被纯 Python 的逐周期行为模型忠实复刻。

# 7

## 模拟总线 DAC 引擎

### 7.1 段表编码

每条 10-bit DAC 总线有一张小的 LUTRAM 段表 (每 tick 组合读), 每段编码:

#### bus segment

```
start_tick / stop_tick : 段的起止 tick (仿射, 带 NUM_SLOTS 个系数, 同  
→ effective_tick)  
start_value / stop_value: 段起止 DAC 码 (10 bit)  
mode                  : edge/hold 或 ramp  
value_select (dual)   : start 与 stop 各一个选择子
```

总线码是**offset-binary**: 码 0 = 负满幅, 码 512 = 真零点 0 V, 码 1023 = 正满幅; 用户层(GUI/API/扫描表) 一律用带符号值 -512..+511 (0 = 0 V), 编译器在生成段/扫描点时自动 +512。引擎的空闲/复位/上电值是 **BUS\_SAFE\_VALUE** (= 中点码 512, 即 0 V), 所以未驱动的总线始终输出 0 V 而不是负满幅。**edge/hold** 段把输出保持在某个值; **ramp** 段在 start 到 stop 之间**线性插值**: 引擎用 Bresenham 整数插值逐 tick 输出  $v(k) = v_{start} \pm \lfloor k \cdot |\Delta|/T \rfloor$  (陡 ramp 每 tick 走多个码), 任意陡度都尽量贴合理想直线并在 stop tick **精确到达目标** (不额外占边沿行)。段 apply 时一个小型组合除法器算出每 tick 基础步长与余数 (仅陡时启用, 缓 ramp 仍是历史的进位 0/1 步进)。**dual value\_select** 让 start 与 stop 各自可以指向一个扫描 slot——于是一条 ramp 能**同时扫两个端点** (扫描-A → 扫描-B), 一条 edge/hold 段能跟随一个扫描的 DAC 码。

### 7.2 模拟 DAC ramp 与扫描端点: 前端脉冲实绘

每条模拟总线是一张小 LUTRAM 段表 (组合读), 段含 start/stop tick(仿射系数)+ start/stop 值 + mode(edge/ramp)+ **双 value\_select**(start 和 stop 各一个)。ramp 段由引擎在本地按 tick 用 Bresenham 整数插值生成 10-bit 阶梯 (陡 ramp 每 tick 走多个码、精确落在目标上, 不占数字边沿行); 双 value\_select 让一条 ramp 的**两个端点各跟一个 scan slot**(scanned-A → scanned-B), edge/hold 段则跟单个被扫 DAC 码。下图是**前端脉冲绘图器实绘**的 DAC 波形。

`host.engine_model.bus_play` 逐 tick 证明了总线/ramp 引擎, 包含被扫描的 ramp 端点。

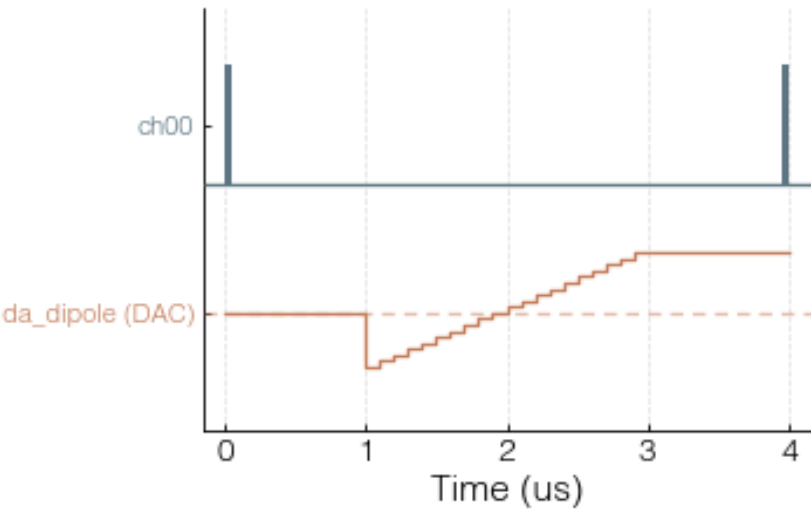


Figure 7.1: 前端脉冲实绘：模拟总线 DAC 波形——保持 0、斜坡 0→1023、保持 1023。引擎在本地按 tick 插值生成 10-bit 阶梯; 双 value\_select 允许斜坡两端各跟一个 scan slot。



# 8

## 三个子系统的深入讨论

本章把两个互相支撑的子系统讲透：**AXI4 突发上传**（怎么把程序映像快速写进 FPGA），以及**逐信号输出延时（事件调度器）**（一个 channel / DAC 总线被延时后到底输出什么）。延时**独立于**仿射扫描引擎：边沿表始终按无延时的标称位置生成，延时只是加在引擎输出上的逐信号事件调度器，把那条 channel / 总线的输出整体往后推  $d$  个 tick。

### 8.1 AXI4 突发上传

主机到 FPGA 的传输是 JTAG-to-AXI: `jtag_axi_0`（一个由 Vivado Tcl 经 JTAG 线驱动的 AXI 主机）→ `axi_bram_ctrl_0` → 顶层后面区段译码的 BRAM 映像。架构上的关键事实是：这两个 IP 都配成**完整 AXI4，而不是 AXI4-Lite**。

**为什么这很重要。**在 JTAG-to-AXI 上，一次单拍写大约要 10 ms（开销在 JTAG 往返，不在数据量）。AXI4-Lite 没有突发，所以每个 32-bit 字都是一次事务；一张 4096 边沿的程序有几千个字，于是**每次 On Pulse 都是数秒级的上传**。完整 AXI4 允许主机每次事务发一个最多 256 拍（AWLEN 上限）的 INCR 突发，于是一段地址连续的字一个往返就写完，整张 4096 边沿映像因此降到约 **100 ms**。

配置的单一真相源在 `fpga/pulse_streamer/create_project.tcl`: `jtag_axi_0` 与 `axi_bram_ctrl_0` 都设 `CONFIG.PROTOCOL {AXI4}`，并把 `M_AXI_ID_WIDTH=1` / `ID_WIDTH=1` 对齐, 32-bit 数据/地址, `SUPPORTS_NARROW_BURST=0`。顶层 `zlc_pulse_streamer_top.v` 把主从 1:1 接线，**包含突发旁带**（`awlen/awsize/awburst/wlast` 等及读通道镜像）。

#### 一个静默退化点

若漂回 AXI4LITE（或丢掉突发旁带），综合仍然成功，但 `-len N` 被忽略，上传又变回数秒级——**不会报错**。契约测试因此断言 `CONFIG.PROTOCOL {AXI4}` 存在、`{AXI4LITE}` 不存在，且主机侧 `m_axi_awlen/m_axi_awburst/m_axi_wlast` 与从机端口映射 `.s_axi_awlen(/.s_axi_awburst(/.s_axi_wlast(` 都已接线。

**主机侧**(`Zou_lab_control/neutral_atom/devices/axi_session.py,VivadoAxiStreamerSession`)。字写入先排成 (`byte_addr, value`) 队列，在 flush 时合并成突发：

- `_burst_runs` 按**插入顺序**（绝不全局排序）扫描待写队列，只合并严格地址连续（步长 4）的项，上限 `burst_max`（默认 256，夹到 `[1, 256]`）。**顺序是有载荷的**：COMMAND 上升沿是对**同一地**

址的两次写 (先 0 后 cmd), `BANK_READY` 的撤销/重置也是对同一地址的两次写; 这些都必须保持有序的单拍写, 所以不会被合并或重排。

- `_write_burst_tcl` 发一条 `create_hw_axi_txn ... -len N -type write -burst INCR`。-burst INCR 是显式写的: Vivado 默认突发类型跨版本不保证是 INCR, 而 FIXED 会把每拍都写到基地址 (静默损坏)。对多字突发, `-data` 是一个拼接的十六进制大数, 其最低位 (最右) 的字落在基地址——所以逐拍的字是高地址在前发出的, 即连续值要逆序拼接。这个字节序是唯一一个容易静默出错的点。
- `_flush` 每个 Vivado 往返里发多个突发 (`write_batch` 限的是突发数, 不是字数), 摊薄主机  $\leftrightarrow$  Tcl 的往返延迟。
- `axi_self_test` 是 warm-start 上电自检: 突发写一段已知 ramp 进扫描 BRAM 区, 再单拍读回比对, 不一致就抛错。它正好抓住两类静默故障——突发 `-data` 字节序错、或仍是 Lite 的 bitstream 忽略了 `-len`——在任何真实脉冲上传之前。

上传不会随扫描点无界增长。点数超过 `2*bank_size` 的扫描, 引擎播 2-bank ping-pong 窗口、主机流式回填 (见前面“无限流式扫描”一章): 在游标 `CURSOR` 之后把空出的 bank 填上下一段并重置 `BANK_READY`。总扫描点数只受主机内存限制, 晚到的回填只会 STALL (`STATUS_UNDERFLOW`), 绝不给出错误点。

## 8.2 逐通道输出延时 (事件调度器 event scheduler)

channel 延时是一条加在引擎输出上的延时, 而不是烤进边沿表里的平移。定义极其简单: 被延时  $d$  个 tick 的 channel, 其延时后输出 `output_delayed[t] = output_undelayed[t-d]`,  $t < d$  之前恒为 0 (fire 之前为 0)。硬件是一个逐信号的事件调度器 (event scheduler): 当引擎那条 channel 的无延时输出翻转时, 把 (翻转时刻 +  $d$ , 新电平) 排进该 channel 的一个小事件 FIFO, 一个自由运行的全局 tick 计数器到时把它弹出到输出寄存器——于是输出整体延后  $d$ 。它只是把那条 channel 的输出整体延后, 因此对任意信号都正确——周期的、非周期的、被 duration 扫描平移过的——都一样。这里没有任何周期性假设, 不“跳整周期”, 没有移位相位/成员关系, 也没有逐扫描点的延时。一个 channel 的延时绝不改变另一个 channel 的时序 (合并是不相交的直通 `out = (state_mask & ~delayed_mask) | delayed_out`)。延时均匀地与 duration 扫描、DAC 扫描、内层 repeat bracket 组合——“延时后的序列”恒等于“把那条 channel 延时  $d$  的无延序列”。它不是扫描量: 延时是每条 channel 的一个固定值, 由主机作为常数携带 (`channel_delays`)。编译它的是 `compile_pulse_table_runtime_program`  $\rightarrow$  `_pulse_table_edge_table` (`sequencer.py`)。

为什么是真实延时, 不是循环。循环视角 (GUI 预览显示的就是它) 会在  $t = 0$  伪造一段“绕回来的尾巴”, 于是 fire 之后的头几个脉冲变成序列尾部的幻影, 破坏真实的实验起始。事件调度器在 fire 时被清零, 所以它让这头几个真实脉冲正确——第一帧是真实的 (在  $t = d$  之前那条 channel 安静、保持 0, 不是循环  $\%T$  的预览样子)。延一个 channel 不会改变任何其它 channel 的周期, 所以  $T$  保持不变。

### output delay (延时 d)

```
cyclic (预览, 仅显示):      [尾巴绕回][..... 原信号右移.....]    <- t=0 处有幻影
event scheduler (硬件, 真相): [.....0.....][原信号右移 d.....]    <- fire 前恒 0,
↪ 第一帧真实
```

机理要点 (排翻转事件、到时弹出; TTL 与 DAC 同源):

- **TTL: 62 路全部可独立延时——事件调度器 (event scheduler)**。TTL 波形是翻转稀疏的, 所以不每拍存一位, 而是存翻转事件: 无延时位在 tick  $t$  翻转时, 向该 channel 的小事件 FIFO (深 `EVT_DEPTH`, 默认 128, 每项 49 bit) 推入 ( $t + d - 1$ , 新电平); 一个自由运行的 48 位全局计数器

`g_time` 等值弹出到输出寄存器——于是 `out[t]=in[t-d]` 精确成立 (首个事件前为 0;  $d = 1$  走一级寄存器;  $d = 0$  直通)。存储随在途翻转数而非延时长度增长: 62 路共  $\sim 2k$  LUT, TTL 延时上限是 32 位字段 ( $\sim 42.9$  s——毫秒级 emCCD 延时直接可用)。每通道 32 位延时值经独立的 DELAY 寄存器区 (`R_DELAY_BASE`, 每 channel 一个 32 位字) 上传。合并仍是不相交直通: 被延时位用调度结果, 其余位原样——延时绝不干扰别的 channel。

- **容量契约: 在途翻转数  $\leq$  `EVT_DEPTH` (默认 128, 可配置)。** 在 tick  $t$  推入的事件在 FIFO 里停留到  $t + d - 1$ , 所以 FIFO 占用 = 该通道自己的  $d$  窗口内的翻转数。主机校验器对完整日程做精确计数——每个扫描点按各自的仿射平移后的边沿时刻、bracket 循环、`repeat_forever` 的回绕缝、以及跨多帧的长窗口全部计入 (早期版本只查单帧、漏掉了扫描缝和  $d >$  帧长的情况); 超限在上硬件前清晰报错。深度可整体重建调整: `streamer_config.json` 的 `evt_fifo_depth` + 顶层参数 `EVT_FIFO_DEPTH` 保持一致即可。
- **延时是物理的。** 写进寄存器的就是用户的原始  $d$  (`repeat_forever` 也一样), 所以**第一帧永远真实**: 被延时通道在 fire 后安静整整  $d$  个 tick 才开始输出, 绝不“提前几个扫描”。代价是无限循环下长延时的在途翻转数按  $\sim$  每扫描翻转数  $\times d/S$  增长 ( $S =$  一次完整扫描的 tick 数) ——这正是下面容量契约要对完整日程精确计数、超限即报错的原因。
- **DAC: 4 条总线各自可延时, 一条总线的 10 位共享一个延时——与 TTL 同源。** 每条 10-bit 总线的**每一个 bit**都是它自己的 1 位事件调度器 (`g_busdly`, 每 bit 深 `BUS_EVT_DEPTH`, 默认 64), 机制与 TTL 完全相同: 值变化时排进事件、 $d$  个 tick 后弹出。一条总线的 10 个 bit**共享同一个**逐总线延时 (`del_bus_ticks`), 所以整条总线 (固定值、被扫值、或 ramp) 被同一个  $d$  整体延后, 而不是每 bit 各延各的。延时值与 TTL 一样是一个 32 位字、走同一个 `R_DELAY` 寄存器区——所以 DAC 与 TTL 的延时**范围相同** (都  $\sim 42.9$  s)。
- **负延时 = 全局平移  $G$ 。** 一条 channel 的  $-d$  等价于其它**所有** channel 都  $+|d|$ 。主机把全局平移  $G = \max(0, -\min(\text{delays}))$  折进每条 channel 与每条总线的有效延时 (TTL 与 DAC 共享**同一个  $G$** ), 使最早的那条信号落在 0、调度器只会看到  $\geq 0$  的延时。零延时即直通。因为 TTL 与 DAC 用同一个 32 位范围、同一套事件调度器, 负延时的全局平移在两者之间**一致**, 绝不会出现“某条信号超范围被拒、另一条没事”的不匹配。
- **DAC 容量契约: 在途值变化数  $\leq$  `BUS_EVT_DEPTH` (默认 64, 可配置 `bus_evt_fifo_depth`)。** 与 TTL 完全同型: 占用 = 该 bit**自己的  $d$  窗口**内的值变化数, 随在途事件数而非延时长度增长, 所以大  $d$  不再受限。唯一现实压力源是一条**被延时的密集 ramp** (每 tick 都变); 超限在 validate 与映像打包时**被清晰报错拒绝** (在上硬件之前, 绝不静默)。
- **`repeat_forever` 倒回边沿 0、重放整张帧——**因为延时是加在输出上的逐通道延时线 (不烘进边沿), 没有需要跳过的“加性延时前导帧”。编译器恒发 `repeat_from_index=0`; RTL (`zlc_edge_streamer.v` 与顶层) 仍保留 `repeat_from_loop_start` 倒回稳态帧的分支, 但对当前实发的程序而言是死路径。

#### 如何证明

预览保持循环 (只为显示)。硬件/编译路径由一个延时真值预言 `delay_line_reference` (`output_delayed[t]=output_undelayed[t-d]`, fire 前为 0) 加上 `host/engine_model.py` 的事件调度器寄存器镜像 (`rtl_delay_line_mirror` / `rtl_bus_delay_line_mirror`) 逐周期模型证明: 编译出的程序过这些周期模型后, 与延时真值**逐 tick 相同** (覆盖 TTL 多 channel、DAC 逐总线、负延时折  $G$ 、与 duration 扫描组合)。容量契约由三层证据锁定: (1) 随机化测试证明解析的在途计数 = 逐拍 push/pop 占用的暴力模拟; (2) 只要解析说“装得下深度  $K$ ”,  $K$  深 FIFO 镜像就逐 tick 复现理想延时; (3) xsim 测试台 `sim/tb_evt_depth.v` 在真 RTL 引擎上验证边界: 恰好  $K$  个在途翻转 = tick 级精确,  $K+2$

个 = 只丢弃多余的 2 个翻转（输出只缺那一个脉冲，绝无污染）。在途事件数超过 `EVT_DEPTH` (TTL) / `BUS_EVT_DEPTH` (DAC) 的延时在 `validate_pulse_streamer_program` 与 `pack_program` 处都被清晰拒绝。

### 8.3 DAC 总线延时与 TTL 完全同源

模拟总线 (DAC) 的延时机制和 TTL **完全相同**：每条总线的**每一个 bit**是它自己的 1 位事件调度器 (`g_busdly`)，值变化时排进事件、 $d$  个 tick 后弹出。一条总线的**10 个 bit 共享一个**逐总线延时 (`del_bus_ticks`)，所以无论这个 DAC 值是**固定值**、**被扫值**，还是**斜坡 (ramp)**，整条总线都被同一个  $d$  整体延后；在  $t < d$  之前总线保持其安全码 (`BUS_SAFE_VALUE`，中点码 = 0 V)。逐总线延时与 TTL 共享**同一个**全局平移  $G$ 、**同一个** 32 位范围 ( $\sim 42.9$  s)、**同一型**容量契约（在途值变化数  $\leq$  `BUS_EVT_DEPTH`，超过即清晰拒绝）。主机把每条总线的延时 tick 数作为常数携带 (`bus_delays`)。

**成本小结。** TTL 每条 channel、DAC 每个 bit 各一个事件 FIFO；这些 FIFO 都是**分布式 RAM (LUTRAM / distributed)**，**不占额外 BRAM** (`ram_style="distributed"`)。每个信号的 32 位延时值走 `R_DELAY` 寄存器区下发，**没有**延时上传映像、**没有**延时 CTRL 字、**没有**迷你装载器。在 35T (`xc7a35t`) 上整套延时落地后仍有余量：LUT 约 59%、BRAM 39/50、DSP 约 58%。

### 8.4 逐通道 clk 输出 (把某通道直接接到时钟)

有时需要某个 TTL 引脚直接输出 FPGA 的 `clk` (例如给外部 DAC 喂时钟)，而引擎按 tick 寄存的输出本身**产生不出**一个时钟方波。为此顶层在引脚映射上加了一层**运行时可选的 clk 复用**：一个 `CHANNEL_COUNT` 位的 `clk_enable` 掩码 (CTRL 字 `C_CLK_ENABLE`，紧接 20 个命令字之后 = 字 20，62 位占 2 个字；CTRL 块里**没有任何延时字**，延时都在 `R_DELAY` 寄存器区) 随程序映像下发，顶层据此把每个引脚选成

#### 逐通道 clk 复用 (`zlc_pulse_streamer_top.v`)

```
assign out_final[n] = clk_en[n] ? ~clk : out[n]; // n:
→ 0..CHANNEL_COUNT-1 (strobe = 反相 clk)
// 引脚映射改为驱动 out_final (不是 out) : assign trig = out_final[6]; assign
→ da_clk0 = out_final[28]; ...
```

所以被标记为 `clk` 的通道，其引脚输出**反相的 clk** (即 `clk` 的**下降沿**对应选通上升沿)，与引擎完全无关。**为什么是反相而不是直接 clk**：DAC 的 40 个并行数据位 (`da_bias_*/da_dipole`) 由引擎在 `posedge clk` 寄存——也就是在**上升沿改变**。若选通就是 `clk`，DAC 会在**同一个上升沿**锁存正在跳变的数据 (而且 `period` 边界上还叠加几十路 TTL 同时翻转的开关噪声)，把一次跳变锁成**半新半旧的第三个码**——这正是“**两个 edge period 之间偶发出现第三种 DA 值**”的根因 (当年靠在两段之间塞一段  $\sim 200$  ms 的 hold 把 DAC 跳变挪离这个忙沿，只是治标)。用**反相 clk**做选通，DAC 改在 `clk` 下降沿锁存 = 数据眼图**正中** (50 MHz 下两侧各  $\sim 10$  ns 已稳定)、且是没有任何东西翻转的**安静半周期**，于是**永远**锁到干净稳定的码。证据见 `sim/tb/_da/_clk/_phase.v` (引擎段跳变逐 tick 干净无中间值；同沿锁存在真实偏斜下会锁进第三个码，眼图正中锁存**从不**)。配套地，主机编译时把该通道的 `mask` 位**强制清 0** (`masks[i] &= ~clk_enable`)，引擎**绝不驱动**它、也就**不会覆盖**`clk` 行为；`clk_enable` 掩码由 `PulseTableState.clk_channels` 经 `clk_enable_mask()` 算出，随 `RuntimeSequenceProgram.clk_enable` 打包进 CTRL 字 (`host.image.CtrlWords.CLK_ENABLE`)。换哪条通道做 `clk` **只是改这份运行时掩码、不需要重新综合**；但首次引入这套 mux 需要重新构建一次 bitstream。DAC 总线成员位不允许设成 `clk` (会破坏 DAC，GUI 已禁用)。这套 mux 是纯组合逻辑、几乎不占资源。

## 8.5 两件没有下放进 sequencer 的事

两条边界澄清，免得把架构搞混：

- **解耦。** sequencer / 边沿流送器**不含**任何相机/触发判断。引擎只播放数字边沿、模拟总线段、和逐通道输出延时线；引擎 HDL (`zlc_edge_streamer.v`) 里**没有**相机/采集/读出/判定逻辑。触发通道只是引擎驱动的又一路数字输出——**何时**计数或阈值判定的决定，住在采集/反馈子系统 (`subsystems / readout`)，不在 timing 里。
- **对齐到 tick (单一来源)。** 时钟只能落在整 tick ( $\geq 20$  ns)，所以字面时间值会被对齐到网格，且只有一个对齐来源 `PulseTableState.snapped()` (`timing/pulse_table.py`): period 持续时间**向下取整到  $\geq 1$  tick** (绝不塌到 0, 例如 5 ns  $\rightarrow$  20 ns); channel 延时与扫描点值**四舍五入到最近 tick** (半值远离零、保号); DAC 扫描点取最近整数码; 而 slot **表达式** (`s0`、`20+s0` 等非数字) **原样保留**——编译器改为对它的仿射 base 取整，所以绑定永不被破坏。它**永不报错** (自动对齐)，对应共聚焦的 `align_to_resolution`。脉冲传输 API 在 `sequencer.timing_payload_to_dict` 里用 `snapped()` 一次性对齐整个状态，GUI 则通过 `align_to_resolution` 背书的分辨率控件逐字段套同一规则——两端共享 `timing/pulse_table.py` 的同一套取整逻辑，所以**所见即所跑**。



# 9

## 容量与资源预算

### 9.1 35T 上解出的容量

容量由 `host.image.solve_capacity(part, channel_count)` 求解（单一真相源），目标  $\leq 90\%$  RAMB36。35T 上解出：

```
solve_capacity('xc7a35tfgg484-2')
```

```
4096 edges + bank_size 2048 (4096 resident) + UNBOUNDED streaming
RAMB36: 39/50 = 78%      LUT: 26%   FF: 12%   DSP: 9%
```

三块并行边沿 BRAM (tick/coeff/mask) + 扫描窗口 BRAM 占了 RAMB36 预算的大头；**总线段表是 LUTRAM (distributed)，不占 RAMB36**——因为总线/ramp 引擎每 tick 组合读它们，搬进 BRAM 会破坏组合读。**延时事件调度器** (TTL 每条 channel 一个深 `EVT_DEPTH` 的事件 FIFO + 每个 DAC bit 一个深 `BUS_EVT_DEPTH` 的事件 FIFO) 同样是分布式 RAM，**不占额外 BRAM**——这正是它能放进 35T 的原因：每个 FIFO 只存在**在途翻转事件**（而非整张序列、也非每拍一格），所以很小。把延时一并算进去后，35T 的整体落地约为 LUT 59%、BRAM 39/50、DSP 58%。Vivado `report_utilization / report_timing_summary` 是最终权威，Python 估算只是预算指引。

### 9.2 64 位 tick 架构评估（何时需要、怎么改、资源够不够）

当前 32 位 tick (20 ns) 封顶的是**单个帧**的时长  $\sim 85.9$  s——不是总运行时长 (repeat forever 无限跑、扫描点数无上限、实验循环不受限)，TTL 延时另有 32 位字段 ( $\sim 42.9$  s，且 repeat forever 下更长的延时自动按周期取模)。真正的长期隐患只有一个：调度器的自由运行计数器 `g_time` 是 48 位，连续运行  $\sim 65$  天后回绕。

若将来确有  $> 85.9$  s 的单帧需求，可行方案是**只加宽基准时间轴**：边沿基准 tick、帧计数器 `tc`、`loop_end` 与各比较器升到 64 位，而**扫描点 slot 值与系数保持 32/16 位**（每个扫描点的仿射偏移量  $\pm 42.9$  s 足够）——这样 `coeff$ \times $slot` 乘积和全部 52 个 DSP 映射**原样不动**，只有末级累加/加法变宽（进位链，不吃 DSP）。代价：tick BRAM  $4096 \times 32 \rightarrow 4096 \times 64$  即 +4 块 RAMB36 (27 $\rightarrow$ 31)、比较器/影子寄存器约 +1k LUT、+1.5k FF；64 位进位链  $\sim 4$  ns，远小于 20 ns 周期，1-tick 播放契约不受影响。CTRL 布局上 `LOOP_END_TICK` 变 LO/HI 两字、映像 tick 字加倍 (pack/unpack/回读校验同步改，一次重建即可，无需兼容)。预计落地：RAMB36 31/50、LUT  $\sim 64\%$ 、DSP 不变——

---

**35T 上可行但目前不需要。**顺手项：下次重建时把 `g_time` 48→64 位 (每通道只多十几个触发器与比较位), 即可消除 65 天回绕。详细论证见 [docs/MAINTAINER\\_NOTES.md](#) §14。

# 10

## 编译与上传流程

### 10.1 build / program / run

FPGA 侧只有一个入口 `fpga\textbackslash build\_and\_program.bat` (内部跑 `create_project.tcl` → `program_fpga.tcl`), server 入口是 `fpga\textbackslash run\_server.bat` (jtag-axi 后端):

#### PowerShell

```
.\fpga\build_and_program.bat --check      # 不连板的 HDL 综合 + 容量自查
.\fpga\build_and_program.bat              # build + program
.\fpga\run_server.bat --check-config      # 打印
↪ project/bit/ltx/xdc/channels/clock/capacity
.\fpga\run_server.bat                    # 启动常驻 hw_axi server (host
↪ 0.0.0.0, port 18861)
```

`create_project.tcl` 生成 `jtag_axi` + `axi_bram_ctrl` + 5 块 BRAM (三块并行边沿 tick/coeff/mask + 扫描窗口 + 总线映像), 并用 `zlc_force_latency2` 把边沿 BRAM 强制成 `READ_LATENCY_B=2`。逐通道/逐总线的输出延时事件调度器是**分布式 RAM (LUTRAM)** 事件 FIFO, **不占额外 BRAM**、也没有延时上传映像。产物 `zlc_pulse_streamer_top.\{bit,ltx\}` 落在 `fpga\textbackslash build\textbackslash ps\textbackslash ps.runs\textbackslash impl_1` (工程用短名 `ps` 是为了让 Vivado 深层的 `run / .Xil` 临时路径不超过 Windows MAX\_PATH 限制, 同时构建仍然留在仓库内的 `fpga\textbackslash build`, 不挪到别处)。server 加载 `.ltx` 是为了在比特流里定位 `jtag_axi` 核 (即 `hw_axi`), 所以 bit 与 ltx 必须来自**同一次编译**。

### 10.2 一次上传里发生了什么

1. 主机校验程序: `validate_pulse_streamer_program` 检查边沿/扫描点/位宽/通道不越界, 并校验**全局有效 tick 单调** (扫描不会把合并后的边沿顺序打乱; 会打乱顺序的扫描被拒绝, 而不是悄悄丢点)。
2. `host.image.pack_program` 把整张程序打包成 32-bit BRAM 映像: CTRL 标量 + 三块并行边沿表 + 2-bank 扫描窗口 + 总线段映像。



3. `prepare`: 发 SAFE → AXI 写映像 → arm 两个扫描 bank (写 `BANK_READY/BANK*_CHUNK`) → 发 LOAD (等 `STATUS_LOADED`)。
4. `fire`: 发 FIRE; 对流式/重复扫描, 启动后台回填线程。
5. `wait_done`: 轮询 STATUS (DONE 位) , 并跟随 CURSOR 回填空出来的 bank。  
`repeat_forever` 永不置 DONE, 所以必须给 timeout 或主动 stop。

#### 全部在硬件前被 Python 验证

`pack_program`↔`unpack_program` 往返、`engine_model` 的逐周期等价 (1-tick 预取 / 边界无缝 / 流式 ping-pong / 总线 ramp) 、以及 `VivadoAxiStreamerSession` 的 `pack`→`upload`→`LOAD`→`fire` 流程 (Tcl 执行器可注入, 无需 Vivado) 都有契约测试覆盖。仓库没有 Verilog 仿真器, 这套 Python 证据是硬件 bring-up 前的正确性保证。